

Helpdesk — Sistema de Tickets de Soporte

Proyecto público de portafolio · Especificación, arquitectura, flujo de trabajo y backlog (Scrum) · Versión 2 (octubre 2026) · Yorkael Mc Carthey Sánchez

1. Resumen y objetivo

Helpdesk es un sistema de gestión de tickets de soporte técnico, en la línea de Zendesk o Freshdesk pero simplificado. El objetivo es un proyecto público que demuestre, con evidencia verificable, arquitectura por capas en .NET, pruebas automatizadas, CI/CD real con GitHub Actions y contenedores con Docker. Es independiente de AgroSystem (SaaS privado y comercial): no reutiliza su código, nombres, dominio ni datos.

2. Stack tecnológico

Capa	Tecnologías
Backend	ASP.NET Core sobre .NET 10 (LTS) , C#, EF Core, PostgreSQL, FluentValidation, Serilog
Autenticación	JWT + refresh tokens; autorización por políticas según rol
Pruebas	xUnit (unitarias en Application) y pruebas de integración con WebApplicationFactory
Frontend	React, TypeScript, Vite, Tailwind CSS, Zustand. Gestor de paquetes: pnpm
DevOps	Docker, docker-compose, GitHub Actions (build y test), Swagger/OpenAPI
Despliegue	Backend en Railway, frontend en Vercel, subdominio helpdesk.yorkael.com

Nota: la versión 1 de este documento indicaba .NET 8. Se cambió a .NET 10 porque .NET 8 y .NET 9 terminan su soporte el 10 de noviembre de 2026, y .NET 10 es LTS con soporte hasta noviembre de 2028.

3. Requisitos funcionales

3.1 Roles

- **Admin:** gestiona usuarios, agentes y categorías; ve todos los tickets.
- **Agente:** ve los tickets asignados o sin asignar, los toma, responde y cambia su estado.
- **Cliente:** crea tickets, ve solo los propios y comenta en ellos.

3.2 Gestión de tickets

- Crear ticket con título, descripción, categoría y prioridad (Baja, Media, Alta, Urgente).
- Estados: Abierto, En progreso, Esperando cliente, Resuelto, Cerrado.
- Asignación a un agente; reasignación permitida.
- Comentarios internos (solo personal) y comentarios visibles para el cliente.
- Historial de cambios por ticket (auditoría): quién cambió qué y cuándo.
- Listado con paginación, filtros (estado, prioridad, agente, categoría) y búsqueda por texto.

3.3 Cuentas y seguridad

- Registro e inicio de sesión con JWT; renovación con refresh token.
- Autorización por políticas según rol en cada endpoint.
- Recuperación de contraseña simulada con enlace de un solo uso (backlog opcional).

4. Requisitos no funcionales

- Errores con respuestas consistentes (problem+json) y manejo global de excepciones.
- Validación de entradas con FluentValidation en todos los endpoints de escritura.

- Logs estructurados con Serilog.
- Pruebas unitarias de la capa Application y al menos 2 endpoints con pruebas de integración.
- CI que compile y corra las pruebas en cada push y pull request.
- Todo el sistema debe levantar con un solo comando: `docker compose up`.
- Sin secretos en el repositorio: variables de entorno y archivo `.env.example`.

5. Arquitectura

Arquitectura por capas. La regla de dependencia permite probar la lógica sin base de datos y reemplazar EF Core o el framework web sin tocar las reglas de negocio.

Proyecto	Responsabilidad
Helpdesk.Api	Controladores REST, middlewares, autenticación, Swagger. Referencia a Application e Infrastructure.
Helpdesk.Application	Casos de uso, DTOs, validadores e interfaces de repositorio. Referencia a Domain.
Helpdesk.Domain	Entidades (Ticket, Comment, User, Category, AuditLog), enums y reglas de negocio. Sin referencias externas.
Helpdesk.Infrastructure	EF Core (Fluent API), repositorios, migraciones, PostgreSQL. Referencia a Application y Domain.
Helpdesk.Tests	Pruebas unitarias (Application) y de integración (Api). Referencia a Application y Api.

6. Modelo de datos

Entidad	Campos principales
User	Id, Name, Email, PasswordHash, Role (Admin/Agent/Client)
Ticket	Id, Title, Description, Status, Priority, CategoryId, CreatedById, AssignedToId, CreatedAt
Category	Id, Name
Comment	Id, TicketId, AuthorId, Content, IsInternal, CreatedAt
AuditLog	Id, TicketId, UserId, Field, OldValue, NewValue, ChangedAt

7. Flujo de trabajo y Git

Se usa GitHub Flow: **main** siempre estable y una rama por historia de usuario (HU), aunque yo sea el único colaborador. Todo el historial (commits, pull requests, issues) va en inglés.

7.1 Ramas y commits

- Ramas: `feature/hu-4-create-ticket`; también `fix/`, `chore/`, `docs/`.
- Dentro de la rama: commits pequeños, en modo imperativo, con Conventional Commits.
- Tipos: `feat`, `fix`, `test`, `docs`, `chore`, `refactor`, `ci`.
`feat(domain): add Ticket entity and status/priority enums`
`test(application): cover ticket creation rules`
`ci: add build and test workflow`

7.2 Pull request y fusión

- Un pull request por HU. El título sigue Conventional Commits y será el mensaje final del commit.
- Descripción con: Summary, Changes, How to test y Closes #N.
- Se fusiona con **Squash and merge**: queda un solo commit por HU en main.
- Al fusionar se borra la rama.

7.3 Sprints y versiones

- Cada sprint es un milestone de GitHub. Al cerrarlo se crea un tag: v0.1.0 (Sprint 1), v0.2.0 (Sprint 2), v1.0.0 (Sprint 3, demo desplegada).
- Protección de main: exigir pull request y, desde el Sprint 3, que pase el CI. No exigir aprobaciones, porque GitHub no permite aprobar el propio PR.

8. Backlog para el GitHub Project

Cada fila es un ítem del Project. Campos: **Title** (en inglés, listo para copiar), **Sprint** y **Points**. Etiquetas sugeridas: backend, frontend, testing, devops, docs. Total: 53 puntos.

ID	Title (issue)	Sprint	Pts	Labels
HU-1	Set up layered solution structure and tooling	1	1	devops
HU-2	Model domain entities and EF Core migrations	1	3	backend
HU-3	Implement registration and JWT authentication	1	5	backend
HU-4	Create tickets with category and priority	1	5	backend
HU-5	List tickets with filters, search and pagination	1	5	backend
HU-6	Assign tickets and change their status	2	3	backend
HU-7	Add public and internal comments to tickets	2	3	backend
HU-8	Expose ticket audit history	2	3	backend
HU-9	Add unit tests for the Application layer	2	5	testing
HU-10	Add integration tests for key endpoints	2	3	testing
HU-11	Containerize the stack with Docker and docker-compose	3	3	devops
HU-12	Add GitHub Actions CI workflow	3	3	devops
HU-13	Build the React UI (login, list, detail)	3	8	frontend
HU-14	Deploy the demo and write the README	3	3	devops, docs

Totales por sprint: Sprint 1 = 19 puntos, Sprint 2 = 17 puntos, Sprint 3 = 17 puntos.

9. Criterios de aceptación por historia

Pegar cada bloque en la descripción del issue correspondiente.

HU-1 · Set up layered solution structure and tooling · Sprint 1 · 1 pt

- La solución compila con `dotnet build`. Proyectos: Api, Application, Domain, Infrastructure y Tests, con las referencias de la sección 5 (Domain sin referencias).
- Todos los proyectos apuntan a `net10.0` y un `global.json` fija el SDK 10.
- Frontend creado con Vite (react-ts), Tailwind y pnpm; `pnpm dev` arranca.
- Existen `.gitignore`, `.env.example` sin valores reales, README inicial en inglés y docs/ con esta especificación.

HU-2 · Model domain entities and EF Core migrations · Sprint 1 · 3 pts

- Entidades User, Ticket, Category, Comment y AuditLog, con enums de estado y prioridad, en Domain.
- DbContext con PostgreSQL y configuración por Fluent API en Infrastructure; Domain no depende de EF Core.
- La migración inicial se aplica sobre una base vacía (PostgreSQL en Docker).

HU-3 · Implement registration and JWT authentication · Sprint 1 · 5 pts

- Endpoints de registro e inicio de sesión; contraseñas con hash.
- JWT con expiración y refresh token; el rol (Admin, Agent, Client) va como claim.
- Políticas de autorización definidas por rol.
- Entradas validadas con FluentValidation; credenciales inválidas devuelven 401 en problem+json.

HU-4 · Create tickets with category and priority · Sprint 1 · 5 pts

- Un cliente crea un ticket con título, descripción, categoría y prioridad; responde 201.
- El ticket nace en estado Abierto y queda ligado al usuario autenticado.
- Un cliente no puede fijar asignado ni estado.
- Se registra la creación en el historial de auditoría.

HU-5 · List tickets with filters, search and pagination · Sprint 1 · 5 pts

- Listado con page, pageSize, estado, prioridad, agente, categoría y búsqueda por texto.
- El cliente ve solo sus tickets; el agente los asignados o sin asignar; el admin todos.
- La respuesta incluye el total y los metadatos de paginación.

HU-6 · Assign tickets and change their status · Sprint 2 · 3 pts

- Un agente toma un ticket sin asignar; el admin puede reasignarlo.
- Las transiciones de estado válidas se definen en Domain; una transición inválida devuelve error claro.
- Cada cambio queda en el historial de auditoría.

HU-7 · Add public and internal comments to tickets · Sprint 2 · 3 pts

- Se puede comentar un ticket; los comentarios internos solo los ve el personal.
- El cliente ve solo comentarios públicos y solo comenta en sus tickets.

HU-8 · Expose ticket audit history · Sprint 2 · 3 pts

- Endpoint con el historial de un ticket: campo, valor anterior, valor nuevo, usuario y fecha, en orden cronológico.
- Acceso restringido al admin.

HU-9 · Add unit tests for the Application layer · Sprint 2 · 5 pts

- Pruebas xUnit de la creación de tickets, las reglas de transición de estado y los validadores.
- No dependen de la base de datos; corren con dotnet test.

HU-10 · Add integration tests for key endpoints · Sprint 2 · 3 pts

- Al menos 2 endpoints (por ejemplo crear y listar tickets) probados con WebApplicationFactory.
- Usan una base de datos desechable y cubren la autenticación.

HU-11 · Containerize the stack with Docker and docker-compose · Sprint 3 · 3 pts

- Dockerfile multi-stage para la API y para el frontend.
- docker-compose con API, PostgreSQL y frontend; docker compose up levanta todo.
- Las variables necesarias están documentadas en .env.example.

HU-12 · Add GitHub Actions CI workflow · Sprint 3 · 3 pts

- En push y pull request a main: restore, build y test del backend; build del frontend con pnpm.
- Badge de CI en el README.
- El chequeo se marca como obligatorio en la protección de main.

HU-13 · Build the React UI (login, list, detail) · Sprint 3 · 8 pts

- Pantallas de inicio de sesión, listado con filtros y paginación, y detalle con comentarios.
- Vistas distintas según el rol; sesión en Zustand.
- Diseño responsivo y manejo de estados de carga y error.

HU-14 · Deploy the demo and write the README · Sprint 3 · 3 pts

- Backend en Railway y frontend en Vercel, accesibles en helpdesk.yorkael.com.
- Datos de ejemplo (seed) para que la demo se vea completa sin registrarse.
- README en inglés con descripción, capturas, diagrama de arquitectura, cómo correrlo y enlace a Swagger.

10. Backlog opcional (sin sprint)

Quedan fuera de los sprints para no inflar el alcance. Se toman solo si sobra tiempo.

ID	Title (issue)	Pts
HU-15	Let admins manage users, agents and categories	5
HU-16	Add simulated password recovery with a one-time link	3

11. Definición de terminado

- Cada HU: pull request fusionado con squash, issue cerrado y tarjeta en Hecho.
- Pruebas pasando y badge de CI en verde.
- El proyecto completo levanta con `docker compose up`.
- Swagger disponible, demo desplegada y sin secretos en el historial de commits.
- Licencia MIT incluida.

12. Configuración del GitHub Project

- Campos: Status (Backlog, En progreso, En revisión, Hecho), Sprint (Sprint 1 a 3) y Points (número).
- Vistas: Table con todo el backlog y Board agrupado por Status para el trabajo diario.
- Default repository: [yorkael/Helpdesk](https://github.com/yorkael/Helpdesk). Cada ítem se convierte en issue para poder enlazarlo con ramas y pull requests.
- Workflows del Project: mover a Hecho al cerrar el issue o fusionar el PR.